Perception for ML-Agents (v0.1)

生成 CV 领域的 ground truth 数据, 包括 (Bounding Box, 语义分割图, 深度图), 并集成 ML-Agents, 作为智能体的观测数据用于强化学习的训练.

集成的 Python 端示例代码可在此获取: Perception-for-ML-Agents-Sample

特点

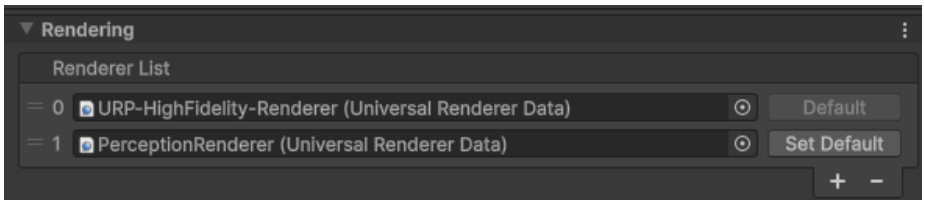
Perception 为Unity ML-Agents提供 ground truth 的 CV 数据. 允许标准的 Unity 相机生成丰富的 Ground Truth 数据, 包括 Bounding Box, 语义分割图, 深度图.

Perception 集成 Unity ML-Agents. 封装这些 ground truth 生成器为自定义的传感器组件, 使强化学习智能体能够直接观察 ground truth. 观测数据会被序列化并通过 ML-Agents API 发送到外部 Python 环境, 从而实现复杂的基于视觉的强化学习模型训练.

Perception 是已弃用的 com.unity.perception 的分支. 剥离了数据集捕获 (离线数据收集) 功能, 专注于轻量级的实时 ground truth 生成, 并支持 **Unity 6**.

安装

在使用的 `Universal Render Pipline Asset` 的 `Rendering -> Renderer List` 中添加 `PerceptionRenderer`.



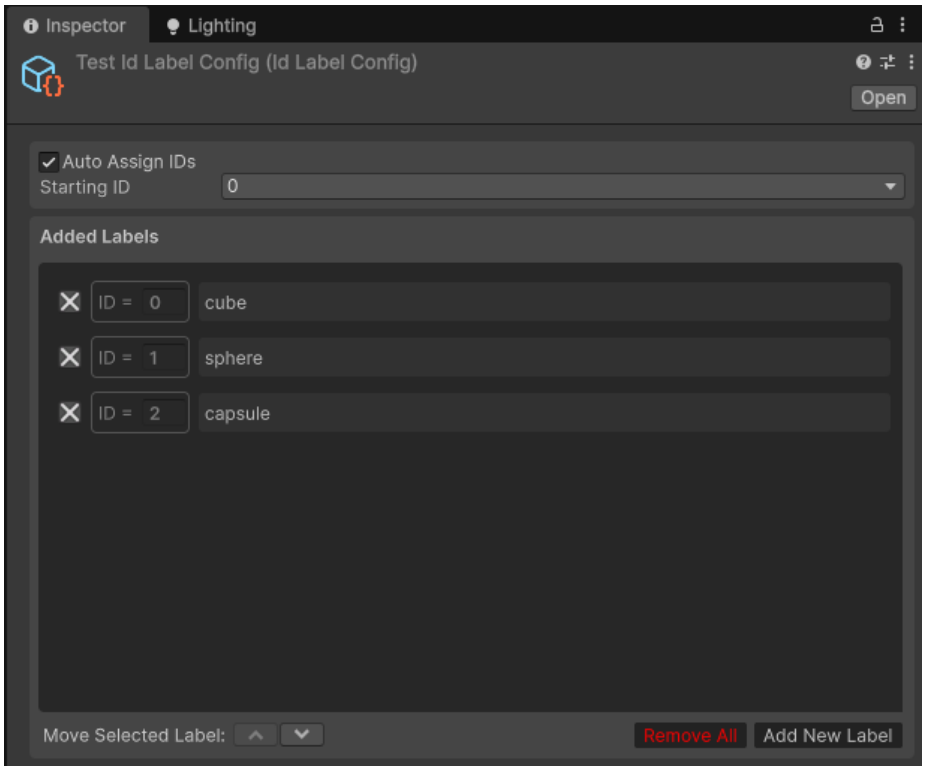
确保项目中已经安装了 `com.unity.ml-agents>=4.0.0` 插件. 安装指南

使用方法

增加标签配置

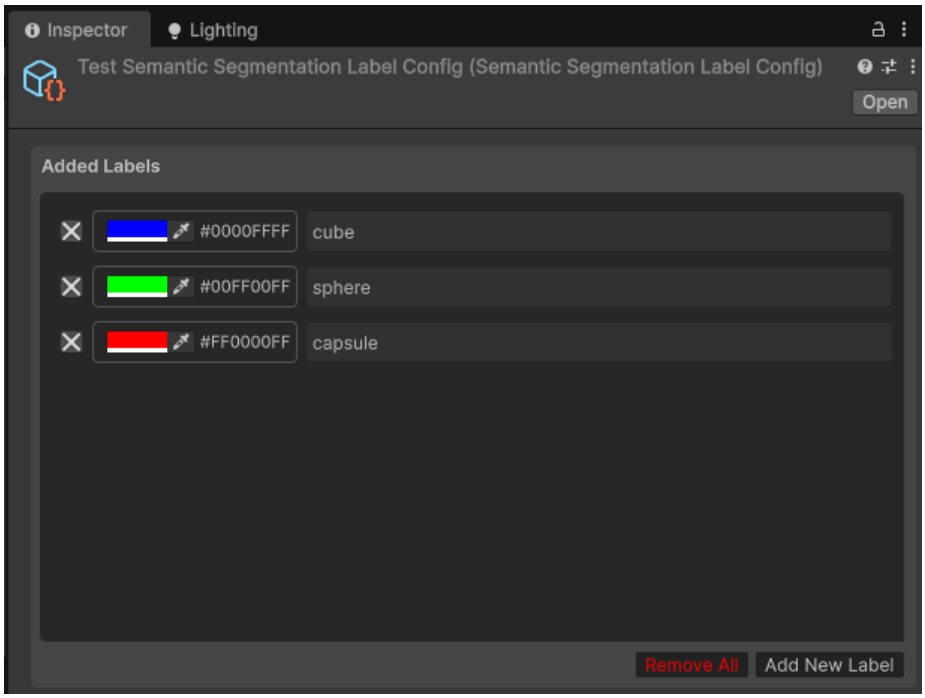
在 `Project` 面板中, `Create -> Perception -> ID Label Config` 创建一个新的 ID 标签配置文件.

可以指定任意的标签名字. 建议将 `Auto Assign IDs` 选项打开, `Starting ID` 设置为 0. 因为 BoundingBox 的物体类别信息是 onehot 类型, 所以 ID 需要从 0 开始连续编号.



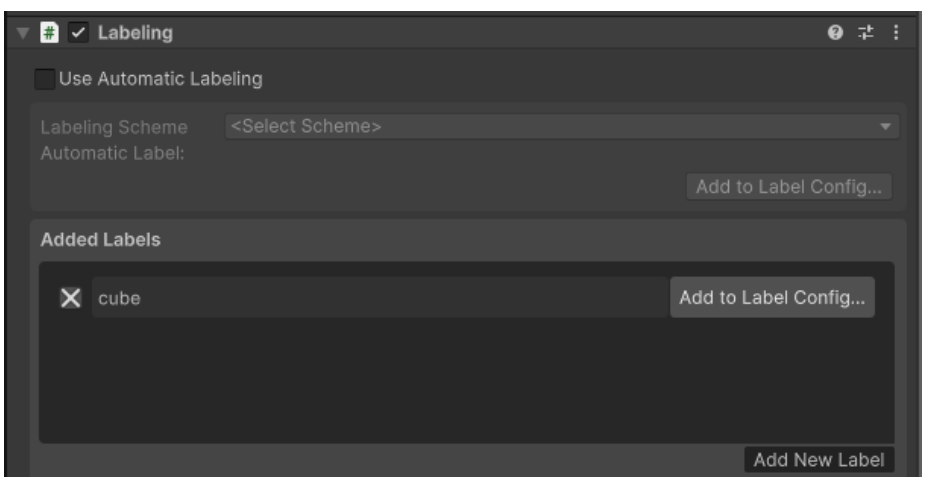
`Create -> Perception -> Semantic Segmentation Label Config` 创建一个新语义分割标签配置文件.

同样可以指定任意标签名字, 并设置任意的语义分割后的颜色.



为 GameObject 添加标签

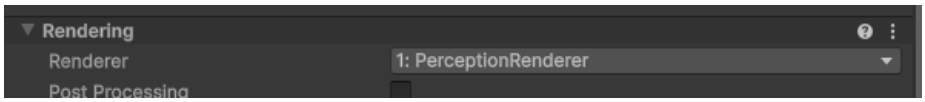
为需要进行感知标注的 GameObject 添加 `Labeling` 组件。点击 `Add New Label` 并输入在 `ID Label Config` 中设置的标签名称。



使用 Perception Camera

将使用 Perception 功能的 Camera 的 `Rendering -> Renderer` 设置为 `PerceptionRenderer`.

注意调整 `Projection -> Clipping Planes` 的 `Near` 和 `Far` 值, 以确保深度图的范围.

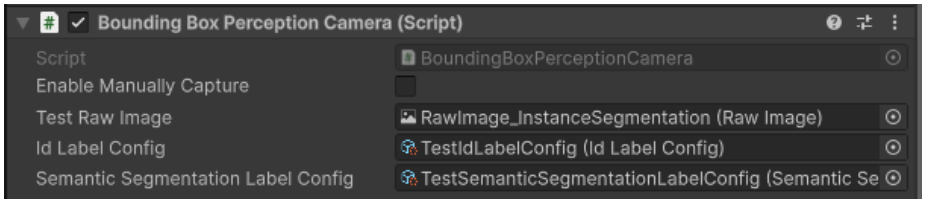


根据需要添加具体的 `Perception Camera` 组件, 可以在同一个 Camera 上添加多个.

Bounding Box Perception Camera

用于生成 BoundingBox, 底层是使用 `Instance Segmentatation` 实现的, 所以需要 ID 标签配置文件.

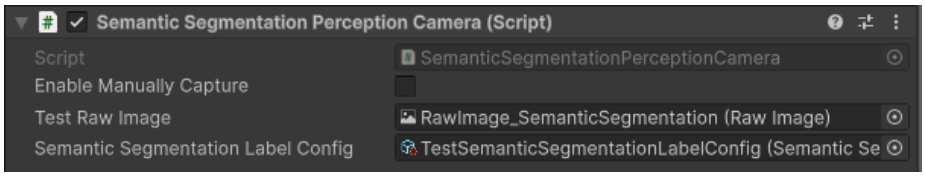
配置参数: * `Enable Manually Capture`: 是否手动调用 Capture 方法进行捕获, 默认每帧自动捕获. ML-Agents 的组件中使用, 通常不需要手动捕获, 设置为默认 `false` 即可. * `Test Raw Image`: 是否在 Canvas 的 `Raw Image` 中显示原始的 Instance Segmentation 图像, 调试或可视化展示用. 显示的 Instance Segmentation 图像会给每个物体分配唯一的颜色, 但不是 ML-Agents 的观测数据, 仅供参考. * `Id Label Config`: ID 标签配置文件. * `Semantic Segmentation Label Config`: 语义分割标签配置文件, 会在 `boundingBoxInfos` 中分配对应的标签所对应的语义分割颜色.



Semantic Segmentation Perception Camera

用于生成语义分割图像, 底层是使用 `Semantic Segmentation` 实现的, 所以需要语义分割标签配置文件.

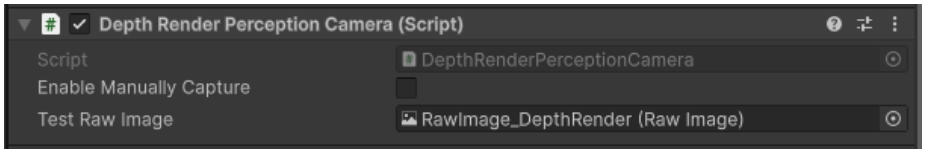
配置参数: * `Enable Manually Capture`: 是否手动调用 Capture 方法进行捕获, 默认每帧自动捕获. ML-Agents 的组件中使用, 通常不需要手动捕获, 设置为默认 `false` 即可. * `Test Raw Image`: 是否在 Canvas 的 `Raw Image` 中显示 Semantic Segmentation, 与传递给 ML-Agents 的观测数据相同. * `Semantic Segmentation Label Config`: 语义分割标签配置文件, 会在 `boundingBoxInfos` 中分配对应的标签所对应的语义分割颜色.



`Depth Render Perception Camera`

用于生成深度图像.

配置参数: * `Enable Manually Capture`: 是否手动调用 Capture 方法进行捕获, 默认每帧自动捕获. ML-Agents 的组件中使用, 通常不需要手动捕获, 设置为默认 `false` 即可. * `Test Raw Image`: 是否在 Canvas 的 `Raw Image` 中显示深度图, 与传递给 ML-Agents 的观测数据相同.



集成到 ML-Agents

与 ML-Agents 提供的传感器组件类似, `Perception` 提供了对应的传感器组件, 可以直接添加到 Agent 上.

`Bounding Box Component`

用于将 Bounding Box 信息作为 `BufferSensor` 观测传递给 ML-Agents.

配置参数:

- `Bounding Box Perception Camera`: 绑定的 `Bounding Box Perception Camera` 组件.
- `Max Number Object`: 最大支持的物体数量, 超过该数量的 Bounding Box 会被忽略.
- `Number Object Type`: 物体类别数量, 应与 ID 标签配置文件中的类别数量一致, 决定了 Bounding Box 信息中 onehot 类别向量的长度, .
- `Random`: 域随机化的比例, 以一定概率随机忽略部分 Bounding Box 或改变 Bounding Box 的尺寸位置, 模拟传感器噪声, 提升训练鲁棒性.
- `Sensor Name`: 传感器名称.
- `Image Size`: 传感器图像尺寸, `X` 为 width, `Y` 为 height.

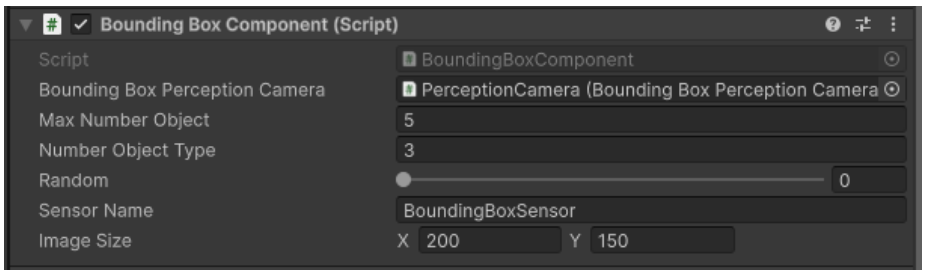
Python 接收到的 observation 格式:

维度为 `(Max Number Object, Number Object Type + 4)` 的二维数组, `Number Object Type + 4` 表示每一个物体的 onehot 类别向量和 Bounding Box 信息.

```
[onehot_id_label_vector..., center_x, center_y, width, height]
```

其中 `center_x, center_y, width, height` 均为归一化到 `[0,1]` 范围内的值, 坐标 `(0,0)` 为图像的左上角, x 轴水平, y 轴垂直, 记录的 `center_x, center_y, width, height` 是 bounding box 的中心点及宽高.

如果识别到的物体数量少于 `Max Number Object`, 则多余的行会被填充为 0.



图中例子所生成的 observation 维度为 `(5, 7)`, 表示最多支持 5 个物体, 类别数量为 3（onehot 向量长度为 3）, 加上 4 个 bounding box 信息, 共 7 维.

`Semantic Segmentation Component`

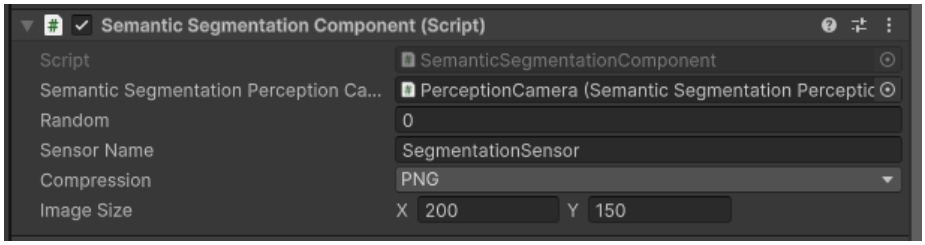
用于将语义分割图像作为 `RenderTextureSensor` 观测传递给 ML-Agents.

配置参数:

- `Semantic Segmentation Perception Camera`: 绑定的 `Semantic Segmentation Perception Camera` 组件.
- `Random`: 域随机化的比例, 以一定概率随机改变物体的中心位置.
- `Sensor Name`: 传感器名称.
- `Compression`: 传感器图像压缩格式.
- `Image Size`: 传感器图像尺寸, `X` 为 width, `Y` 为 height.

Python 接收到的 observation 格式:

维度为 `(3, height, width)` 的三通道图像, 每个像素点的 RGB 值对应语义分割标签配置文件中设置的颜色值. 三通道全 `1` 的像素表示背景, 没有识别到任何物体.



图中例子所生成的 observation 维度为 `(3, 150, 200)`.

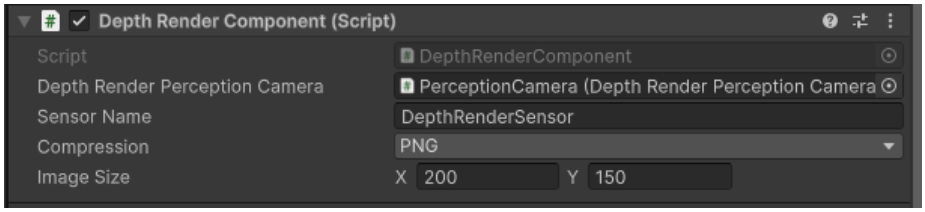
`Depth Render Component`

用于将深度图像作为 `RenderTextureSensor` 观测传递给 ML-Agents.

配置参数: * Depth Render Perception Camera: 绑定的 Depth Render Perception Camera 组件. * Sensor Name: 传感器名称. * Compression: 传感器图像压缩格式. * Image Size: 传感器图像尺寸, X 为 width, Y 为 height.

Python 接收到的 observation 格式:

维度为 (1, height, width) 的一通道图像, 每个像素点为 [0,1] 范围内的灰度值, 表示深度信息, 0 为近裁剪面, 1 为远裁剪面.



图中例子所生成的 observation 维度为 (3, 150, 200).

Python 端接收 observation

在 Python 端, ML-Agents 会将 Perception 传感器生成的观测数据作为 Agent 的观测数据进行接收和处理. 具体的 observation 维度和格式请参考上面的各个传感器组件的说明.

可以参考提供的 Perception_unity_wrapper.ipynb Jupyter Notebook文件, 了解如何在 Python 端接收和处理这些观测数据.

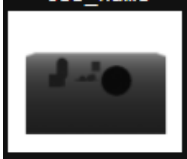
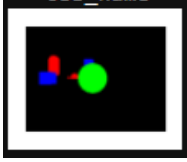
以下是 Perception_unity_wrapper.ipynb 的部分简要示例说明:

```
{'TestPerceptionAgent?team=0': ['BoundingBoxSensor', 'DepthRenderSensor', 'SegmentationSenso
{'TestPerceptionAgent?team=0': [(5, 7), (1, 150, 200), (3, 150, 200)]}
```

表示 TestPerceptionAgent?team=0 这个 Agent 有 3 个传感器, 分别是 BoundingBoxSensor、DepthRenderSensor 和 SegmentationSensor, 对应的 observation 维度分别为 (5, 7)、(1, 150, 200) 和 (3, 150, 200).

以下是第 3 帧的观测数据示例:

```
========== step 3 ==========
[ma_name] : TestPerceptionAgent?team=0
--obs_name-- : BoundingBoxSensor (5, 7)
[[[1.         0.         0.         0.45       0.35333332 0.07000002
   0.06666666]
  [0.         0.         1.         0.19999999 0.38333333 0.08
   0.19333333]
  [1.         0.         0.         0.1575     0.5        0.125
   0.12      ]
  [0.         0.         1.         0.33499998 0.48000002 0.08000001
   0.05333331]
  [0.         1.         0.         0.475      0.5        0.20999998
   0.27999997]]]
--obs_name-- : DepthRenderSensor (1, 150, 200)
```



```
--obs_name-- : SegmentationSensor (3, 150, 200)
```